Chapter 21 — Interview Question Bank

How to use this bank

This chapter collects the Java questions that come up again and again in real interviews — gathered from big public question lists (GitHub collections, LeetCode Discuss interview-experience posts, Java 8 stream-coding lists) — and worded the way interviewers usually say them.

- ⋆ marks a question that appears in many sources: expect it.
- Each answer is short enough to **say out loud** in under a minute. Where this book teaches the topic in full, a → **More** link takes you there. Chapter 19 §19.29 already answers 42 core questions in more depth; those entries here are deliberately brief and link to it.
- Questions the rest of the book does not cover still get a complete short answer here, so nothing is missing.
- Every "what does this print" output and every coding answer was run on a real JDK; the printed results are real.
- **How to drill:** cover the answer, say yours out loud, then uncover and compare. Mark the ones you missed and repeat them the next day.

Versions: the book targets **Java 25 LTS** (LTS = long-term support release). Features are tagged with the Java version that made them final.

1. Platform and language basics

Q1. ⋆ What is the difference between JDK, JRE and JVM? The **JVM** (Java Virtual Machine) runs bytecode. The **JRE** (Java Runtime Environment) was JVM + standard libraries, enough to run programs. The **JDK** (Java Development Kit) adds the compiler `javac` and tools (`jshell`, `jcmd`, `jlink`…). Since Java 11 Oracle no longer ships a separate JRE (some vendors, such as Eclipse Temurin, still offer JRE builds); you can build a trimmed runtime for your app with `jlink`. → *More:* §1.3 JDK, JRE and JVM

Q2. Why is Java platform independent? Is the JVM platform independent? `javac` compiles source to **bytecode** (`.class` files), a portable instruction set; any JVM on any OS runs the same bytecode ("write once, run anywhere"). The JVM itself is *not* platform independent — each OS/CPU pair has its own JVM build. → *More:* §1.6 Bytecode and class files

Q3. Is Java a pure object-oriented language? No. It has eight **primitive** types (`int`, `double`…) that are not objects, and `static` members that belong to no object. Everything else is an object, and wrappers (`Integer`) let primitives act as objects.

Q4. ⋆ What are the primitive types and their sizes? `byte` 8 bits, `short` 16, `char` 16 (unsigned UTF-16 code unit), `int` 32, `long` 64, `float` 32, `double` 64, `boolean` (true/false, size not specified). Integer types are signed two's complement. Default field values are `0`, `0.0`, `'\u0000'`, `false`; local variables have no default and must be assigned before use. → *More:* §2.3 Primitive types

Q5. Primitive vs reference types? A primitive variable holds the value itself; a reference variable holds a pointer to an object on the heap (or `null`). Copying a reference copies the pointer, so both variables see the same object. Primitives can't be `null` and can't be used as generic type arguments. → *More:* §2.5 Reference types and null

Q6. ⋆ What are wrapper classes and autoboxing? What can go wrong? Each primitive has a wrapper object (`Integer`, `Long`, `Character`…). **Autoboxing** converts `int → Integer` automatically; **unboxing** does the reverse. Pitfalls: unboxing `null` throws `NullPointerException`; `==` on wrappers compares references (works by luck inside the −128…127 cache); boxing in hot loops allocates objects. → *More:* §7.6 Wrapper classes and autoboxing

Q7. ⋆ Is Java pass-by-value or pass-by-reference? Always pass-by-value; for objects the value copied is the reference, so a method can mutate the caller's object but not make the caller's variable point elsewhere. → *More:* §4.2 · §19.29 Q6

Q8. Widening vs narrowing conversion; what is casting? Widening (`int → long → double`) is automatic and safe (except precision loss for big `long → float/double`). **Narrowing** (`double → int`, `int → byte`) needs an

explicit **cast** `(int) 3.99` and may truncate or overflow: `(byte) 200` is `-56`. In expressions, `byte` / `short` / `char` are promoted to `int`, which is why `short s = 1; s = s + 1;` doesn't compile but `s += 1` does (compound assignment casts implicitly). → *More:* §2.7 Type conversion

Q9. What does `var` do, and where can't you use it? `var` (Java 10) lets the compiler infer a **local** variable's type from its initializer; the variable is still statically typed. Not allowed for fields, method parameters, return types, without an initializer, or with a bare `null` initializer. Use it when the type is obvious from the right-hand side. → *More:* §2.6 var

Q10. ⋆ What does `static` mean? Can a static method use instance members? A static member belongs to the class, one copy shared by all instances. A static method has no `this`, so it can't use instance fields or methods directly (it needs an object reference). Static blocks run once when the class is initialised. → *More:* §5.5 Static members · §19.29 Q41

Q11. ⋆ Explain `final` vs `finally` vs `finalize`. `final` = can't reassign / override / extend; `finally` = block that always runs after `try`; `finalize()` = deprecated-for-removal GC callback (JEP 421, Java 18) — use try-with-resources or `Cleaner`. → *More:* §5.8 final · §19.29 Q7

Q12. What are the access modifiers? `private` (same class only), *package-private* (no keyword: same package), `protected` (same package plus subclasses in other packages), `public` (everyone). Top-level classes can only be `public` or package-private. Since Java 9, modules can additionally hide whole packages. → *More:* §5.6 Access modifiers

Q13. Why is `main` `public static void main(String[] args)`? What changed in Java 25? The launcher must call it without creating an object (`static`), from outside the package (`public`), and expects no result (`void`). Java 25 (JEP 512) allows **instance main methods** and **compact source files**: a file containing just `void main() { IO.println("Hi"); }` runs with `java Hello.java`. → *More:* §4.9 main and simpler main

Q14. What is the difference between `this` and `super`? `this` refers to the current object (disambiguate fields, call another constructor with `this(...)`, pass yourself). `super` reaches the parent's version of a member (`super.toString()`) or calls a parent constructor with `super(...)`. Before Java 25, `this(...)` / `super(...)` had to be the first statement; Java 25's flexible constructor bodies (JEP 513) allow statements before it (validation, computing arguments) as long as they don't read fields or call instance methods; the only use of `this` allowed there is assigning the class's own fields that have no initializer. → *More:* §6.1 Inheritance and super

Q15. What happens on integer overflow? How do you detect it? It silently wraps around: `Integer.MAX_VALUE + 1 == Integer.MIN_VALUE`. Use `Math.addExact` / `multiplyExact` (throw `ArithmeticException`), a wider type (`long`), or `BigInteger`. → *More:* §2.10 Integer overflow

Q16. Why shouldn't you use `double` for money? What do you use instead? Binary floating point can't represent most decimal fractions exactly (`0.1 + 0.2` prints `0.30000000000000004`). Use `BigDecimal` created from a **string** or `BigDecimal.valueOf(double)` (not `new BigDecimal(0.1)`, which captures the binary error), with an explicit `RoundingMode`, or store amounts as `long` cents. → *More:* §7.7 BigInteger and BigDecimal

2. OOP: classes, inheritance and polymorphism

Q17. ⋆ What are the four principles of OOP? Encapsulation — hide state behind methods so invariants hold. **Abstraction** — expose *what* an object does, not *how* (interfaces, abstract classes). **Inheritance** — a subclass reuses and specialises a parent. **Polymorphism** — one call, different behaviour depending on the object's run-time class. → *More:* §5.7 Encapsulation · LLD Handbook

Q18. ⋆ Abstraction vs encapsulation? Abstraction is a *design* idea: model only the essential behaviour and hide complexity behind a simple contract. Encapsulation is the *mechanism*: private fields, controlled access through methods. Abstraction decides what to show; encapsulation enforces what to hide.

Q19. ⋆ What is polymorphism? Compile-time vs run-time? Compile-time (static) polymorphism is **overloading**, resolved by the compiler from argument types. Run-time (dynamic) polymorphism is **overriding**: `Animal a = new Dog(); a.speak()` runs `Dog.speak()` because the JVM dispatches on the actual object's class. → *More:* §6.4 Polymorphism and casting

Q20. ⋆ Overloading vs overriding? Overloading: same name, different parameters, chosen at compile time. Overriding: same signature in a subclass, chosen at run time; can't reduce visibility, can't throw

broader checked exceptions, may return a subtype (**covariant return**). → *More:* §6.3 Overriding · §19.29 Q12

Q21. ⋆ Can you override a static method? A private method? Can you overload `main`? Static: no — a same-signature static method in a subclass **hides** the parent's; which runs depends on the reference's declared type. Private: no — it isn't inherited, so a same-named subclass method is unrelated. `main`: yes, you can overload it, but the launcher only calls the standard signature. → *More:* §6.2 Members, hiding and Object

Q22. ⋆ Why doesn't Java support multiple inheritance of classes? How do you get it? Inheriting state and implementations from two parents creates the "diamond problem" (which parent's field or method wins?). Java allows one superclass but many interfaces. If two interfaces provide the same `default` method, the class must override it and may pick one with `A.super.method()`. → *More:* §6.6 Interfaces

Q23. ⋆ Interface vs abstract class (and what changed in Java 8/9)? Interfaces: multiple implementation, no instance state, `default`/`static` methods (8) and `private` methods (9). Abstract class: single inheritance, fields, constructors. Interface for a capability, abstract class for shared state among related classes. → *More:* §6.5 · §19.29 Q9

Q24. Can an abstract class have a constructor? Can it have no abstract methods? Yes to both. The constructor runs when a subclass is created (via `super(...)`), initialising shared state. A class can be `abstract` with zero abstract methods simply to forbid direct instantiation.

Q25. ⋆ Composition vs inheritance — which do you prefer? Inheritance ("is-a") couples the subclass to the parent's implementation details; a change in the parent can break it (the "fragile base class" problem). Composition ("has-a") holds another object in a field and delegates to it, so you can swap implementations and expose only what you choose. Prefer composition; use inheritance only for a true is-a relation designed for extension. → *More:* LLD Handbook

Q26. Association vs aggregation vs composition? **Association**: objects know each other. **Aggregation**: a "has-a" where the part can live without the whole (a team has players). **Composition**: the part's lifetime is owned by the whole (an order owns its order lines).

Q27. What is a constructor? Default constructor? Constructor chaining? A constructor initialises a new object; it has the class name and no return type. If you write none, the compiler adds a no-arg **default constructor**; if you write any, it doesn't. Chaining: `this(...)` calls another constructor of the same class, `super(...)` a parent one; if neither is written, the compiler inserts `super()`, so a parent without a no-arg constructor causes a compile error. → *More:* §5.3 Constructors

Q28. What is a private constructor used for? To stop outside instantiation: utility classes with only static methods, singletons, and classes that hand out instances through static factory methods (`List.of`, `Integer.valueOf`).

Q29. What is the initialisation order of a class? Once per class: static field initialisers and static blocks, top to bottom (parent class first). Per object: parent constructor chain first, then the child's instance field initialisers and instance blocks in order, then the rest of the child constructor body. So a parent constructor that calls an overridden method sees the child's fields still at default values (see Q218). → *More:* §5.10 Initialisation order

Q30. What methods does `Object` have? `equals`, `hashCode`, `toString`, `getClass`, `clone` (protected), `wait`/`notify`/`notifyAll` (thread coordination on the object's monitor), and the deprecated `finalize`. Every class inherits them. → *More:* §6.13 equals, hashCode and toString

Q31. What is an enum, and why is it better than int constants? An `enum` is a class with a fixed set of instances: type-safe (you can't pass 7 where a `Day` is expected), can have fields, constructors, methods and per-constant behaviour, works in `switch`, and comes with `values()`, `valueOf()`, `EnumMap`, `EnumSet`. Enum constants are singletons, safe even across serialization. → *More:* §6.9 Enums

Q32. Static nested vs inner class vs anonymous class? A **static nested** class has no link to an outer instance (prefer it). An **inner** (non-static) class holds a hidden reference to its outer object — it can use the outer fields but can leak memory if it outlives the outer. An **anonymous** class is an unnamed one-off implementation, mostly replaced by lambdas for single-method interfaces. → *More:* §6.10 Nested classes

Q33. What is a marker interface? Give examples. An interface with no methods that tags a class for special treatment: `Serializable`, `Cloneable`, `RandomAccess`. Today annotations usually do this job, but a

marker interface is a real type, so the compiler can check it.

Q34. What is a covariant return type? An overriding method may declare a *subtype* of the parent's return type, e.g. `Animal reproduce()` overridden as `Dog reproduce()`, which saves callers a cast.

Q35. What is upcasting and downcasting? When do you get `ClassCastException`? Upcasting (`Animal a = dog`) is implicit and always safe. Downcasting (`(Dog) a`) is explicit and checked at run time; if the object isn't really a `Dog` it throws `ClassCastException`. Guard with pattern matching: `if (a instanceof Dog d) d.bark();`. → *More:* §3.5 Pattern matching for instanceof

Q36. What are coupling and cohesion? Coupling is how much one class depends on another's details — aim low. **Cohesion** is how focused a class is on one responsibility — aim high. Low coupling plus high cohesion makes code easier to change and test.

Q37. ⋆ What are the SOLID principles? Single responsibility (one reason to change), **O**pen/closed (extend without editing), **L**iskov substitution (subtypes must work wherever the parent is expected), **I**nterface segregation (small focused interfaces), **D**ependency inversion (depend on abstractions, inject concrete classes). → *More:* LLD Handbook

Q38. ⋆ How do you write a thread-safe singleton? Best: an `enum` with one constant (serialization- and reflection-safe), or the **holder idiom** (a nested static class holds the instance; the JVM's class initialisation makes it lazy and thread-safe). Double-checked locking also works but the field **must** be `volatile`. Code in Q248.

3. Strings

Q39. ⋆ Why is `String` immutable? Safe sharing between threads, safe as a hash key (hash cached), poolable, and secure for values that are checked then used (paths, class names). → *More:* §7.1 · §19.29 Q1

Q40. ⋆ What is the String pool? How many objects does `new String("hi")` create? A JVM table (on the heap since Java 7) with one shared instance per literal; `new String("hi")` makes one new object plus the pooled literal if not already there — "one or two". `intern()` returns the pooled copy. → *More:* §7.1 · §19.29 Q2

Q41. ⋆ `String` vs `StringBuilder` vs `StringBuffer`? Immutable vs mutable-unsynchronised vs mutable-synchronised (legacy). Build strings in loops with `StringBuilder`. → *More:* §7.3 · §19.29 Q3

Q42. ⋆ `==` vs `equals()` for strings? `==` asks "same object?", `equals` asks "same characters?". Literals happen to share a pooled object, so `==` sometimes "works" — a classic trap; always use `equals` (or `"x".equals(var)` to be null-safe). See Q210. → *More:* §7.1 Strings and the pool

Q43. Why is `+=` on a `String` in a loop slow? Each `+=` copies the whole string into a new object, so n appends cost $O(n^2)$ character copies. Outside loops, `+` is fine: since Java 9 the compiler emits an `invokedynamic` call to `StringConcatFactory`, which builds the result efficiently. → *More:* §7.3 StringBuilder

Q44. How is a `String` stored internally? Since Java 9 (**compact strings**, JEP 254), as a `byte[]` plus a coder flag: Latin-1 strings use one byte per character, others UTF-16 (two bytes). The hash code is computed lazily and cached in a field. → *More:* §7.1 Strings and the pool

Q45. Why prefer `char[]` over `String` for passwords? A `String` can't be wiped and may linger in memory (and in the pool) until GC, where heap dumps can expose it; a `char[]` can be overwritten with `Arrays.fill(pw, '\0')` after use. That's why `Console.readPassword()` returns `char[]`.

Q46. `trim()` vs `strip()`; what else did Java 11 add? `trim()` removes characters ≤ U+0020 (ASCII spaces and control chars); `strip()` (11) removes all Unicode whitespace. Java 11 also added `isBlank()`, `lines()`, `repeat(n)`, `stripLeading/Trailing()`. → *More:* §7.2 The String API

Q47. What does `intern()` do, and when would you use it? Returns the canonical pooled instance with the same content, adding it to the pool if absent. Rarely needed: it can save memory when you hold millions of duplicate strings, but `-XX:+UseStringDeduplication` (G1 since Java 8u20, the other collectors since Java 18) usually does that for you.

Q48. What are text blocks? Multi-line string literals (Java 15) delimited by `"""`; incidental indentation is stripped, so JSON/SQL/HTML can be pasted readably. They are ordinary `String`s. → *More:*

§2.12 Strings and text blocks

Q49. How do you split, join and format strings? `"a,b".split(",")` (regex; trailing empty strings dropped), `String.join(",", list)`, `Collectors.joining(", ", "[", "]")`, `StringJoiner`, and `"%s=%d".formatted(k, v)` (Java 15) or `String.format`. → *More:* §7.4 Formatting

Q50. Is `String` thread-safe? Is `StringBuilder`? `String` yes (immutable). `StringBuilder` no — don't share one between threads; give each thread its own, or use `StringBuffer` in the rare case one buffer really is shared.

4. equals, hashCode, immutability and cloning

Q51. ⋆ What is the `equals`/`hashCode` contract? What breaks if you override only `equals`? Equal objects must have equal hash codes; override only `equals` and `HashSet`/`HashMap` stop finding "equal" objects because they hash to different buckets (Q224 shows it). → *More:* §6.13 · §19.29 Q4

Q52. How do you write a correct `equals`? Check identity (`this == o`), then type (`o instanceof Point p`, or `getClass() == o.getClass()` if subclasses must never equal the parent), then compare significant fields (`Objects.equals` for references, `==` for primitives except `Double.compare`/`Float.compare`). Make the class final or use `getClass()` to stay symmetric with subclasses. Build `hashCode` from the same fields: `Objects.hash(x, y)`. Records generate both. → *More:* §6.13 equals and hashCode

Q53. Can two unequal objects have the same hash code? Yes — that's a **collision** and is legal (`"Aa"` and `"BB"` both hash to 2112). Collisions only cost performance; the map falls back to `equals`. → *More:* §6.13 equals and hashCode

Q54. ⋆ How do you make a class immutable? `final` class, `private final` fields, no setters, defensive copies in and out (`List.copyOf`), no `this` escaping. Records cover most of it, shallowly. → *More:* §5.9 Immutability basics · §19.29 Q8

Q55. What are the benefits of immutable objects? Thread-safe without locks, safe as map keys and set elements, can be cached and shared freely, easier to reason about (no state changes after construction), and failure-atomic (an exception can't leave them half-updated). The cost is allocating a new object for every "change".

Q56. Shallow copy vs deep copy? A shallow copy duplicates the top object but shares the objects its fields point to; a deep copy recursively copies them too. `Object.clone()` and `new ArrayList<>(list)` are shallow.

Q57. Why is `Cloneable`/`clone()` considered broken? What do you use instead? `Cloneable` has no `clone` method (it just switches on `Object.clone`'s field-by-field copy), `clone` is protected, returns `Object`, bypasses constructors, is shallow, and clashes with `final` fields. Prefer a **copy constructor** (`new Point(other)`) or a static factory (`Point.copyOf(p)`).

Q58. `Comparable` vs `Comparator`? `compareTo` defines one natural order inside the class; `Comparator` objects define any number of external orders. Never compare by subtraction (overflow); use `Integer.compare` or `Comparator.comparing`. → *More:* §10.8 · §19.29 Q19

5. Exceptions

Q59. ⋆ Explain the exception hierarchy. `Throwable` has two branches: `Error` (serious JVM problems: `OutOfMemoryError`, `StackOverflowError` — don't catch) and `Exception`. Under `Exception`, `RuntimeException` and its subclasses are **unchecked**; all other exceptions are **checked**. → *More:* §8.2 The Throwable hierarchy

Q60. ⋆ Checked vs unchecked exceptions? Checked must be caught or declared (recoverable conditions like `IOException`); unchecked signal bugs (`NullPointerException`, `IllegalArgumentException`) and needn't be declared. → *More:* §8.3 · §19.29 Q23

Q61. Error vs Exception? An `Error` means the JVM or environment is in trouble (out of memory, stack overflow, class-linkage failure); the application usually can't recover, so you let it propagate. An `Exception` is a condition the program can reasonably handle. → *More:* §8.2 The Throwable hierarchy

Q62. `throw` vs `throws`? `throw` is a statement that throws one exception object now: `throw new IllegalStateException("closed")`. `throws` is part of a method signature listing checked exceptions the

method may pass to its caller. → *More:* §8.7 Throwing exceptions

Q63. ⋆ How does try-with-resources work? Closes every `AutoCloseable` declared in the header in reverse order; a close failure after a body failure is attached as a suppressed exception (Q228 shows the order). → *More:* §8.6 · §19.29 Q24

Q64. ⋆ When does `finally` not run? What if both `try` and `finally` return? Only on `System.exit`, JVM crash/kill, or a thread that never leaves the `try`. A `return` in `finally` overrides the `try`'s return and swallows any exception — never do it (Q212). → *More:* §8.4 · §19.29 Q25

Q65. Is `try` without `catch` allowed? Yes: `try`–`finally`, or try-with-resources with no `catch` at all. A `try` alone, with neither `catch` nor `finally` nor resources, doesn't compile. → *More:* §8.4 try/catch/finally

Q66. Why must `catch` blocks go from specific to general? What is multi-catch? A `catch (Exception e)` before `catch (IOException e)` makes the second unreachable — a compile error. Multi-catch `catch (IOException | SQLException e)` handles unrelated types in one block; the variable is implicitly final, and the alternatives can't be subclasses of each other. → *More:* §8.5 Multi-catch

Q67. How do you create a custom exception? Checked or unchecked? Extend `RuntimeException` (unchecked, the common modern choice) or `Exception` (checked, when callers must handle it), and provide constructors that take a message and a **cause** so the original exception is kept (`super(msg, cause)`). → *More:* §8.7 Custom exceptions

Q68. Can an overriding method throw a broader checked exception? No. It may throw the same checked exceptions, narrower ones, or none, and any unchecked exceptions — otherwise callers using the parent type would face exceptions they didn't expect. → *More:* §6.3 Overriding

Q69. `ClassNotFoundException` vs `NoClassDefFoundError`? `ClassNotFoundException` is a checked exception from code that loads a class *by name* (`Class.forName`, a class loader) and can't find it. `NoClassDefFoundError` is an `Error`: a class that was present at compile time is missing at run time, or its static initialiser failed earlier.

Q70. What is exception chaining, and why does it matter? Wrapping a low-level exception in a higher-level one while keeping it as the **cause**: `throw new OrderException("save failed", e)`. The stack trace then shows "Caused by: …", so you don't lose the root cause. → *More:* §8.8 Chaining

Q71. What are exception-handling best practices? Don't swallow exceptions (empty `catch`); catch only what you can handle; keep the cause; log *or* rethrow, not both; prefer specific exceptions; validate arguments early (`Objects.requireNonNull`); don't use exceptions for normal control flow; clean up with try-with-resources. → *More:* §8.10 Best practices

6. Generics

Q72. Why use generics? Compile-time type safety (a `List<String>` can't receive an `Integer`), no casts on read, and reusable algorithms that work for many types. → *More:* §9.1 Why generics

Q73. ⋆ What is type erasure? Generic types are checked by the compiler, then erased to their bounds with casts inserted; at run time `List<String>` and `List<Integer>` are the same class. So no `new T()`, no `new T[]`, no `instanceof List<String>`, no `List<int>`. → *More:* §9.7 · §19.29 Q26

Q74. ⋆ Explain `? extends T` vs `? super T` (PECS). Producer `extends` (read as `T`, can't add), consumer `super` (can add `T`, reads give `Object`). → *More:* §9.6 · §19.29 Q27

Q75. Is `List<Integer>` a subtype of `List<Number>`? No — generics are **invariant**. Otherwise you could add a `Double` to a list of integers through the `List<Number>` reference. Use `List<? extends Number>` to accept both read-only. Arrays, by contrast, are covariant and fail at run time with `ArrayStoreException`. → *More:* §9.6 Invariance and wildcards

Q76. What is a bounded type parameter? `<T extends Comparable<T>> T max(List<T> xs)` restricts `T` to types that have `compareTo`, so the method can call it. Multiple bounds: `<T extends Number & Comparable<T>>` (class first). → *More:* §9.5 Bounded type parameters

Q77. What is a raw type, and why avoid it? Using a generic class without type arguments (`List list`) — kept for pre-Java-5 compatibility. It turns off type checks, so a wrong element blows up later with a `ClassCastException` far from the bug. → *More:* §9.8 Erasure restrictions

Q78. Can you overload `void f(List<String>)` and `void f(List<Integer>)`? No. After erasure both become `f(List)` — same signature, compile error ("name clash"). → *More:* §9.8 Erasure restrictions

7. Collections

Q79. Explain the Collections Framework hierarchy. `Iterable` → `Collection` → `List` (ordered, indexed: `ArrayList`, `LinkedList`), `Set` (no duplicates: `HashSet`, `LinkedHashSet`, `TreeSet`), `Queue`/`Deque` (`ArrayDeque`, `PriorityQueue`). `Map` (`HashMap`, `LinkedHashMap`, `TreeMap`) is separate because it stores key–value pairs, not single elements. Java 21 added `SequencedCollection`/`SequencedMap` for types with a defined order. → *More:* §10.1 The hierarchy

Q80. ⋆ How does `HashMap` work internally? Bucket array (16, load factor 0.75), index = spread hash `& (n−1)`, collisions chained, bucket becomes a red-black tree beyond 8 entries (when the table has ≥ 64 buckets), table doubles when size > capacity × load factor; lookup compares hash then `equals`. → *More:* §10.5 HashMap internals · §19.29 Q13

Q81. Why is `HashMap` capacity a power of two? What does "spreading" the hash do? With n a power of two, `hash & (n − 1)` is a cheap replacement for `hash % n`. Because that only uses the low bits, `HashMap` XORs the high 16 bits into the low 16 (`h ^ (h >>> 16)`) so keys whose hash codes differ only in high bits still land in different buckets. On resize each entry either stays at index *i* or moves to *i* + *oldCapacity*. → *More:* §10.5 HashMap internals

Q82. What happens when two keys have the same hash code? They go to the same bucket; `put` walks the bucket and uses `equals` to decide "same key → replace value" or "different key → add a node". `get` does the same. Many collisions degrade a bucket from O(1) to O(n) (O(log n) once treeified). → *More:* §10.5 HashMap internals

Q83. Can `HashMap` have a null key? `Hashtable`? `ConcurrentHashMap`? `TreeMap`? `HashMap`: one null key (stored in bucket 0) and null values. `Hashtable` and `ConcurrentHashMap`: no nulls at all. `TreeMap`: no null key with natural ordering (it calls `compareTo` → NPE), null values are fine. → *More:* §10.5 HashMap internals

Q84. Why must map keys be immutable? What if you mutate a key? The entry stays in the bucket of the old hash, so lookups miss it. → *More:* §10.5 · §19.29 Q14

Q85. ⋆ `HashMap` vs `Hashtable` vs `synchronizedMap` vs `ConcurrentHashMap`? Unsynchronised with nulls / legacy fully synchronised / one lock wrapper / fine-grained concurrent with atomic compound operations. → *More:* §14.12 Concurrent collections · §19.29 Q15

Q86. ⋆ How does `ConcurrentHashMap` achieve thread safety? How did Java 7 differ? Java 8+: lock-free reads, CAS into empty buckets, `synchronized` on a bucket's head node for updates, cooperative resizing; Java 7 used 16 lock "segments". Iterators are weakly consistent. → *More:* §14.12 · §19.29 Q16

Q87. ⋆ Is `map.get(k) == null ? map.put(k, v) : …` safe on a `ConcurrentHashMap`? No — it's a check-then-act race: two threads can both see `null` and both put. Use the atomic methods: `putIfAbsent`, `computeIfAbsent(k, key -> new ArrayList<>())`, `merge(word, 1, Integer::sum)`. Keep the functions short — they run while the bucket is locked. → *More:* §14.12 Concurrent collections

Q88. ⋆ `ArrayList` vs `LinkedList`? Array-backed, O(1) index access, cache-friendly vs doubly linked, O(1) only at ends or an iterator. Default to `ArrayList`. → *More:* §10.2 · §19.29 Q17

Q89. How does `ArrayList` grow? `new ArrayList<>()` starts with a shared empty array and allocates 10 slots on the first `add`. When full it grows to about 1.5× (`old + old/2`) and copies the elements. Each copy is O(n) but rare, so `add` is **amortised O(1)**. Pass an initial capacity if you know the size. → *More:* §10.2 Lists

Q90. `ArrayList` vs `Vector`? `Vector` is the legacy synchronised version (every method locked, grows 2×). Don't use it; use `ArrayList`, or `CopyOnWriteArrayList`/`Collections.synchronizedList` when shared. → *More:* §10.2 Lists

Q91. `HashSet` vs `LinkedHashSet` vs `TreeSet`? How does `HashSet` work internally? `HashSet` is unordered O(1); `LinkedHashSet` keeps insertion order; `TreeSet` keeps elements sorted, O(log n), and supports `floor`/`ceiling`/ranges. Internally a `HashSet` *is* a `HashMap` whose keys are the elements and whose values are one shared dummy object; `add` returns `false` if the key was already there. → *More:* §10.3 Sets

Q92. ⋆ `HashMap` vs `LinkedHashMap` vs `TreeMap`? How is `TreeMap` implemented? Unordered / insertion- or access-ordered / sorted. `TreeMap` is a **red-black tree** (a self-balancing binary search tree), so `get` / `put` / `remove` are O(log n) and it offers `floorKey`, `ceilingKey`, `headMap`, `tailMap`, `firstKey`. It uses `compareTo` / `Comparator`, not `equals`, to detect duplicates. → *More:* §10.4 Maps · §19.29 Q18

Q93. ⋆ Fail-fast vs fail-safe iterators? How do you remove while iterating? Fail-fast iterators throw `ConcurrentModificationException` on structural change through another path (tracked by `modCount`); concurrent collections' iterators are weakly consistent or snapshot-based. Remove with `Iterator.remove()` or `removeIf`. → *More:* §10.7 · §19.29 Q20

Q94. `Comparable` vs `Comparator` — how do you sort a list of employees by salary, then name? `list.sort(Comparator.comparingDouble(Employee::salary).thenComparing(Employee::name))`; add `.reversed()` for descending. `Collections.sort` and `List.sort` use a stable merge sort (TimSort) for objects. → *More:* §10.8

Q95. `List.of` vs `Arrays.asList` vs `Collections.unmodifiableList`? Truly unmodifiable and null-hostile / fixed-size view that writes through / read-only view of a list that may still change. → *More:* §10.10 · §19.29 Q22

Q96. What is `CopyOnWriteArrayList` and when do you use it? A thread-safe list that copies the whole underlying array on every write; reads and iteration take no lock and see a snapshot (no `ConcurrentModificationException`). Great for read-mostly data such as listener lists; terrible for frequent writes. → *More:* §14.12 Concurrent collections

Q97. `Queue`, `Deque`, `PriorityQueue` — and why prefer `ArrayDeque` to `Stack`? `Queue` is FIFO (`offer` / `poll` / `peek`), `Deque` adds both ends (stack or queue), `PriorityQueue` is a binary heap that always yields the smallest element (O(log n) `offer` / `poll`). `Stack` is synchronised legacy extending `Vector` (so it also exposes index access that breaks the stack abstraction); `ArrayDeque` is unsynchronised, faster, and gives `push` / `pop` / `peek` through the `Deque` interface. → *More:* §10.6 · §19.29 Q21

Q98. What are `EnumMap`, `WeakHashMap` and `IdentityHashMap`? `EnumMap`: a compact array indexed by enum ordinal — fastest map for enum keys. `WeakHashMap`: keys held by weak references, so an entry disappears once nothing else references the key (used for metadata caches). `IdentityHashMap`: compares keys with `==` instead of `equals` (graph traversal, serialization).

Q99. What Java 8 methods did `Map` gain? `getOrDefault`, `putIfAbsent`, `computeIfAbsent`, `computeIfPresent`, `compute`, `merge`, `forEach`, `replaceAll`, `remove(k, v)`. Word count in one line: `counts.merge(word, 1, Integer::sum)`. → *More:* §10.4 Maps

Q100. What are sequenced collections (Java 21)? New interfaces `SequencedCollection`, `SequencedSet`, `SequencedMap` give every ordered collection uniform `getFirst()`, `getLast()`, `addFirst/Last`, `removeFirst/Last` and `reversed()` — so `list.getLast()` replaces `list.get(list.size() - 1)`. → *More:* §10.11 Sequenced collections

Q101. What is the time complexity of common collection operations? `ArrayList`: get O(1), add-at-end amortised O(1), insert/remove middle O(n), `contains` O(n). `HashMap` / `HashSet`: O(1) average. `TreeMap` / `TreeSet`: O(log n). `PriorityQueue`: offer/poll O(log n), peek O(1). `ArrayDeque`: O(1) at both ends. → *More:* §10.12 Big-O cheat sheet · §19.7

Q102. ⋆ How would you implement an LRU cache? `LinkedHashMap` in access order with `removeEldestEntry` returning `size() > capacity` (Q249). Without it: a `HashMap` from key to node plus a doubly linked list — move a node to the front on access, evict from the tail; both O(1). Make it thread-safe with a lock, or use a library like Caffeine in production. → *More:* §10.4

8. Lambdas, functional interfaces and Optional

Q103. ⋆ What are the main features of Java 8? Lambdas and method references, functional interfaces (`java.util.function`), the Stream API, `Optional`, `default` and `static` interface methods, the `java.time` date/time API, `CompletableFuture`, new `Map` methods, and Metaspace replacing PermGen.

Q104. What is a lambda expression? A compact way to write an implementation of a functional interface: `(a, b) -> a + b`. It's not an anonymous class: it has no own `this` (it uses the enclosing one), and the JVM creates it through `invokedynamic`, usually without a new class file. → *More:* §11.2 Lambda syntax

Q105. ⋆ What is a functional interface? Is `@FunctionalInterface` required? An interface with exactly one abstract method (`default`/`static` methods and `Object`'s public methods don't count). The annotation is optional; it makes the compiler reject a second abstract method. → *More:* §11.2 · §19.29 Q30

Q106. Name the core functional interfaces. `Function<T,R>` (T → R), `BiFunction<T,U,R>`, `Supplier<T>` (() → T), `Consumer<T>` (T → nothing), `Predicate<T>` (T → boolean), `UnaryOperator<T>` (T → T), `BinaryOperator<T>` ((T,T) → T), plus primitive versions like `IntPredicate`, `ToIntFunction` that avoid boxing. → *More:* §11.3

Q107. Supplier vs Consumer? A `Supplier<T>` takes nothing and produces a value (`get()`), e.g. a lazy default in `orElseGet`. A `Consumer<T>` takes a value and returns nothing (`accept()`), e.g. the action in `forEach`. → *More:* §11.3 Built-in functional interfaces

Q108. What are method references? Which kinds exist? Shorthand for a lambda that only calls a method: static (`Integer::parseInt`), bound instance (`System.out::println`, `str::startsWith`), unbound instance (`String::length` — the first argument becomes the receiver), and constructor (`ArrayList::new`). → *More:* §11.4 Method references

Q109. Why must variables used in a lambda be effectively final? The lambda captures a copy of the value; allowing later reassignment would make the copy and the variable silently disagree, and lambdas may run on other threads. → *More:* §11.5

Q110. ⋆ What is `Optional`, and how should you use it? A container that holds one value or none, used as a **return type** for "maybe no result" instead of returning `null`. Consume it with `map`, `filter`, `orElse`, `orElseGet`, `orElseThrow`, `ifPresentOrElse`. `Optional.of(null)` throws NPE; use `ofNullable`. → *More:* §11.7 Optional

Q111. ⋆ `orElse` vs `orElseGet`? Common misuses of `Optional`? `orElse(x)` always evaluates `x`; `orElseGet(() -> x)` only when empty — matters for expensive defaults. Misuses: `get()` without a check, `Optional` fields/parameters, `Optional<List>`, returning `null` from an `Optional` method. → *More:* §11.7 · §19.29 Q29

Q112. What are `default` methods for? Can a default method be overridden? They let an interface gain new methods without breaking existing implementations — that's how `Collection.stream()` and `Iterable.forEach` were added in Java 8. Implementations can override them; static interface methods can't be overridden and are called as `Interface.method()`. → *More:* §6.6 Interfaces

9. Streams

Q113. What is a stream? Collection vs Stream? A stream is a one-shot pipeline of operations over a data source: it stores nothing, is lazy, doesn't modify its source, and can be consumed only once (a second terminal operation throws `IllegalStateException`). A collection stores elements and can be iterated many times. → *More:* §12.1 What a stream is

Q114. ⋆ Intermediate vs terminal operations? What does "lazy" mean? Intermediate operations (`filter`, `map`, `sorted`, `distinct`, `limit`) return a new stream and do nothing yet. A terminal operation (`collect`, `forEach`, `reduce`, `count`, `findFirst`) starts the work. Elements flow one at a time through the whole pipeline, and short-circuiting operations stop early (Q223). → *More:* §12.2 Laziness

Q115. ⋆ `map` vs `flatMap`? `map` turns each element into exactly one element (`Stream<List<Integer>>` stays a stream of lists). `flatMap` turns each element into a stream and concatenates them (`lists.stream().flatMap(List::stream)` → `Stream<Integer>`). → *More:* §12.3 Intermediate operations

Q116. How do you create a stream? `collection.stream()`, `Stream.of(a, b)`, `Arrays.stream(arr)`, `IntStream.range(0, n)`, `Stream.iterate(seed, next)`, `Stream.generate(supplier)`, `Files.lines(path)`, `string.chars()`, `Pattern.splitAsStream`. → *More:* §12.1 What a stream is

Q117. What does `collect` do? Name the useful collectors. `collect` is a mutable reduction into a container. Collectors: `toList()`, `toSet()`, `toMap(k, v, merge)`, `joining(sep)`, `groupingBy(classifier, downstream)`, `partitioningBy(predicate)`, `counting()`, `summingInt`, `averagingDouble`, `mapping`, `collectingAndThen`, `teeing` (Java 12). → *More:* §12.5 Collectors

Q118. ⋆ `groupingBy` vs `partitioningBy`? `groupingBy` splits into any number of groups by a key (`Map<K, List<T>>`); `partitioningBy` splits into exactly two by a predicate (`Map<Boolean, List<T>>`, both keys always present). → *More:* §12.5 Collectors

Q119. `Collectors.toList()` vs `Stream.toList()`? `stream.toList()` (Java 16) returns an **unmodifiable** list and allows nulls. `Collectors.toList()` currently returns an `ArrayList` but promises nothing about mutability.

Use `Collectors.toCollection(ArrayList::new)` when you need to modify the result. → *More:* §12.5 Collectors

Q120. ⋆ What happens with duplicate keys in `Collectors.toMap`? It throws `IllegalStateException` ("Duplicate key"). Pass a merge function: `toMap(Emp::dept, Emp::salary, Double::sum)`; add a map supplier such as `LinkedHashMap::new` to control the map type. → *More:* §12.5 Collectors

Q121. What does `reduce` do? Combines all elements into one value with an associative function: `nums.stream().reduce(0, Integer::sum)`. Without an identity it returns `Optional<T>`. For sums prefer `mapToInt(...).sum()` (no boxing). → *More:* §12.4 Terminal operations and reduce

Q122. `findFirst` vs `findAny`; `anyMatch`/`allMatch`/`noneMatch`? `findFirst` respects encounter order; `findAny` may return any element (cheaper in parallel). The match operations short-circuit and return `boolean`; `allMatch` on an empty stream is `true`, `anyMatch` is `false`. → *More:* §12.4 Terminal operations

Q123. ⋆ Are parallel streams always faster? When do you use them? No. They split work over the common `ForkJoinPool`; worth it only for large, CPU-bound, easily splittable sources (arrays, `ArrayList`, ranges) with independent, stateless operations. Bad for I/O, small data, `LinkedList`/`iterate` sources, ordered operations like `limit`, and shared mutable state (adding to a shared `ArrayList` from `forEach` loses elements or throws). → *More:* §12.8 Parallel streams

Q124. Why have `IntStream`, `LongStream`, `DoubleStream`? To avoid boxing every number into an object. They add `sum`, `average`, `max`, `summaryStatistics`, `range`. Convert with `mapToInt`, back with `boxed()`. → *More:* §12.6 Primitive streams

Q125. `forEach` vs `forEachOrdered`; `peek`? In parallel streams `forEach` may process in any order; `forEachOrdered` keeps encounter order (slower). `peek` is for debugging only — it runs only if a terminal operation pulls elements through it. → *More:* §12.8 Parallel streams

Q126. What are stream gatherers (Java 24)? `Stream.gather(Gatherer)` lets you write custom intermediate operations, the way collectors are custom terminal ones. Built-ins in `Gatherers`: `windowFixed(3)`, `windowSliding(3)`, `fold`, `scan`, `mapConcurrent`. → *More:* §12.9 Stream gatherers

10. Modern Java (9 → 25)

Q127. ⋆ What are the main features from Java 9 to 25? 9: modules, `List.of`, JShell. 10: `var`. 11: `String` helpers, `HttpClient`, single-file launch. 14: switch expressions. 15: text blocks. 16: records, `instanceof` patterns, `Stream.toList`. 17: sealed classes. 21: virtual threads, pattern matching for `switch`, record patterns, sequenced collections, generational ZGC. 22: unnamed `_`, Foreign Function & Memory API. 24: gatherers, no `synchronized` pinning. 25: compact source files, module imports, flexible constructor bodies, scoped values. → *More:* §17.2 The version table · §19.29 Q42

Q128. ⋆ What is a record? Can it have extra fields or extend a class? A class whose whole job is to hold a fixed set of values: the compiler writes the constructor, the accessors (`name()`, not `getName()`), and `equals`/`hashCode`/`toString`. Implicitly final, extends `Record` (so it can't extend another class), fields are `private final` (shallowly immutable), no extra instance fields (static ones are fine); can implement interfaces and validate in a compact constructor. → *More:* §6.8 Records · §19.29 Q10

Q129. Record vs Lombok `@Data` vs a normal class? A record is a language feature: final, immutable, fixed shape, no setters, works with record patterns. Lombok `@Data` generates getters, setters, `equals`/`hashCode` for a mutable class through an annotation processor. Use a normal class (or Lombok) when you need mutability, inheritance or hidden state (e.g. JPA entities). → *More:* §6.8 Records

Q130. ⋆ What are sealed classes and why use them? `sealed ... permits A, B` limits subclasses; each must be `final`, `sealed` or `non-sealed`. The compiler then knows all cases, so `switch` over the type can be exhaustive without `default`. → *More:* §6.7 · §19.29 Q11

Q131. ⋆ What is pattern matching for `instanceof` and `switch`? `if (obj instanceof String s)` tests and binds in one step (Java 16). Switch patterns (Java 21) match on type, with guards and null handling:

```java
String describe(Object o) {
    return switch (o) {
        case null -> "null";
        case Integer i when i > 0 -> "positive int";
        case Integer i -> "int";
        case String s -> "string of " + s.length();
        default -> "other";
    };
}
```

→ *More:* §3.6 Pattern matching for switch

Q132. What are record patterns? Patterns that deconstruct a record in place (Java 21): `if (p instanceof Point(int x, int y))` or `case Circle(Point(var x, var y), double r) ->`. With sealed types they give algebraic-data-type style code. Java 22's unnamed pattern `_` skips components you don't need. → *More:* §3.7 Record patterns

Q133. Switch statement vs switch expression? The arrow form (`case A -> ...`) has no fall-through. A switch **expression** yields a value (`int n = switch (d) { ... };`), must be exhaustive, and uses `yield` inside a block. The old colon form falls through without `break`. → *More:* §3.3 switch expressions

Q134. ⋆ What are virtual threads? JVM-managed lightweight threads mounted on a few carrier threads; blocking I/O unmounts them, so millions of blocking tasks are cheap. Help I/O-bound concurrency, not CPU-bound speed. Don't pool them. → *More:* §15.4 Virtual threads · §19.29 Q34

Q135. What is pinning, and is it still a problem? A virtual thread is **pinned** when it can't unmount from its carrier while blocked, so it ties up an OS thread. Before Java 24, blocking inside `synchronized` pinned; JEP 491 (Java 24) fixed that. It still pins when blocking under a native frame (a `native` method or FFM call that calls back into Java) and when blocking during class loading or class initialisation. Diagnose with the JFR event `jdk.VirtualThreadPinned`. → *More:* §15.5 Pinning and JEP 491

Q136. Virtual threads vs reactive programming (WebFlux, RxJava)? Both serve many concurrent I/O operations with few OS threads. Reactive code does it with non-blocking callbacks and operator chains — efficient, but hard to read and debug. Virtual threads keep simple blocking code and normal stack traces. Reactive still fits streaming and back-pressure use cases.

Q137. What are scoped values (Java 25)? `ScopedValue` passes immutable context (user, request ID) down a call chain for a bounded scope — `ScopedValue.where(USER, u).run(task)` — without parameters. Unlike `ThreadLocal` it can't be changed, is unbound automatically when `run` / `call` returns, and child threads forked in a `StructuredTaskScope` inherit it without copying. Final in Java 25 (JEP 506). → *More:* §15.6 ThreadLocal and scoped values

Q138. What is structured concurrency? Is it final? It treats a group of subtasks as one unit: fork them in a scope, join, and if one fails the others are cancelled — no leaked threads. Still a **preview** in Java 25 (fifth preview, JEP 505), so it needs `--enable-preview`. → *More:* §15.7 Structured concurrency

Q139. What are Java modules (JPMS)? Java 9's module system: a `module-info.java` declares what a module `requires` and which packages it `exports`. It gives strong encapsulation (non-exported packages are inaccessible, even by reflection unless `opens`), reliable configuration, and `jlink` custom runtimes. Java 25 (JEP 511) adds `import module java.base;` to import all packages a module exports. → *More:* §16.12 Modules

Q140. Why upgrade from Java 8/11/17 to 21 or 25? Language productivity (records, patterns, text blocks), virtual threads, better GCs (generational ZGC, G1 improvements), lower memory (compact object headers, a product option in 25 enabled with `-XX:+UseCompactObjectHeaders`), faster start-up (the AOT cache, Java 24–25), security and support lifetime (LTS), and frameworks such as Spring Boot 3+ requiring Java 17+. → *More:* §17.12 Upgrading

11. Concurrency

Q141. Process vs thread? A **process** is a running program with its own memory space; a **thread** is a path of execution inside a process, sharing the heap with other threads but having its own stack. Threads are cheaper to create and communicate through shared memory — which is why they need synchronisation. → *More:* §14.1 Concurrency basics

Q142. ⋆ How do you create a thread? `Runnable` vs extending `Thread`? Implement `Runnable` / `Callable` and hand it to a `Thread` or, better, an `ExecutorService`; Java 21 adds `Thread.ofPlatform()` / `Thread.ofVirtual()` builders. Prefer `Runnable` over extending `Thread`: it separates the task from the mechanism and leaves your class free to extend something else. → *More:* §14.2 Creating threads

Q143. ⋆ `start()` vs `run()`? `start()` creates a new thread that then calls `run()`. Calling `run()` directly just executes the method on the current thread — no concurrency (Q232). Calling `start()` twice throws `IllegalThreadStateException`. → *More:* §14.2 Creating threads

Q144. What are the thread states? `NEW`, `RUNNABLE` (running or ready), `BLOCKED` (waiting for a monitor lock), `WAITING` (`wait()`, `join()`, `park` with no timeout), `TIMED_WAITING` (`sleep`, timed `wait` / `join`), `TERMINATED`. → *More:* §14.3 Thread lifecycle

Q145. ⋆ `sleep()` vs `wait()`? `Thread.sleep` pauses the current thread and keeps any locks it holds. `obj.wait()` must be called while holding `obj`'s monitor (else `IllegalMonitorStateException`), **releases** it, and waits until `notify` / `notifyAll`, a timeout, or interruption. Always call `wait` in a `while` loop that re-checks the condition (spurious wake-ups). → *More:* §14.8 wait/notify

Q146. `notify()` vs `notifyAll()`? `notify` wakes one arbitrary waiter; `notifyAll` wakes all, and each re-checks its condition. Prefer `notifyAll` unless all waiters wait for the same condition and any one can proceed — a wrong `notify` can wake the "wrong" thread and stall everyone. → *More:* §14.8 wait/notify

Q147. Why are `wait` / `notify` in `Object` and not in `Thread`? Because every object has a monitor and the waiting is on a *condition protected by that object's lock*, not on a particular thread. → *More:* §14.8 wait/notify

Q148. What is `join()`? What does `yield()` do? `t.join()` blocks the caller until thread `t` finishes (and guarantees its writes are visible). `Thread.yield()` hints that the scheduler may run another thread — rarely useful. → *More:* §14.3 Thread lifecycle

Q149. How do you stop a thread? Cooperatively: call `t.interrupt()`, and have the task check `Thread.currentThread().isInterrupted()` or catch `InterruptedException` from blocking calls and exit. If you catch `InterruptedException` and can't exit, restore the flag with `Thread.currentThread().interrupt()`. `Thread.stop()` is gone (throws `UnsupportedOperationException` since Java 20). → *More:* §14.4 Interruption

Q150. ⋆ What is a race condition? Why isn't `count++` thread-safe? A race condition is when correctness depends on the timing of threads. `count++` is three steps — read, add, write — so two threads can read the same value and one increment is lost. Fix with `synchronized`, `AtomicInteger.incrementAndGet()`, or `LongAdder`. → *More:* §14.5 Race conditions

Q151. ⋆ How does `synchronized` work? Method vs block; static synchronized? Every object has an intrinsic lock (**monitor**). A synchronized instance method locks `this`; a static synchronized method locks the `Class` object; a block locks the object you name (prefer a private lock object and the smallest block). The lock is **reentrant** (the owning thread can re-acquire it) and gives visibility on release/acquire. → *More:* §14.6 synchronized

Q152. ⋆ What does `volatile` do? Is it enough for a counter? Visibility and ordering of reads/writes of that field; no atomicity, so not enough for `count++`. Fine for a stop flag or a published reference. → *More:* §14.7 volatile · §19.29 Q31

Q153. What is the Java Memory Model / happens-before? The rules that say when one thread's writes must be visible to another: unlock → lock, volatile write → read, `start()`, `join()`, program order. Without such an edge, reordering and caching can hide writes. → *More:* §15.8 · §19.29 Q32

Q154. ⋆ `synchronized` vs `ReentrantLock`? `ReentrantLock` does the same mutual exclusion but adds `tryLock()` (with timeout), interruptible locking, optional fairness, and several `Condition`s per lock. You must `unlock()` in `finally`. Use `synchronized` by default (simpler, can't forget to release); use `ReentrantLock` for those extras. → *More:* §14.9 Explicit locks

Q155. What are `ReadWriteLock` and `StampedLock`? `ReentrantReadWriteLock` lets many readers in at once but only one writer — useful when reads dominate. `StampedLock` (Java 8) adds **optimistic reads**: read without locking, then `validate(stamp)` and retry under a real lock if a write happened. `StampedLock` isn't reentrant. → *More:* §14.9 Explicit locks

Q156. What are atomic classes and CAS? `AtomicInteger`, `AtomicLong`, `AtomicReference` update a single variable without locks using **CAS** (compare-and-set: "set to new value only if it still equals the

expected value", one CPU instruction, retried on failure). Under heavy contention on a counter, `LongAdder` scales better by spreading updates over cells. → *More:* §14.10 Atomic variables

Q157. What is the ABA problem? A CAS sees value A, another thread changes it to B and back to A, and the CAS succeeds even though the state changed in between. Fix with a version stamp: `AtomicStampedReference`.

Q158. ⋆ What is a deadlock? How do you prevent and detect it? Threads holding locks while waiting for each other's locks forever. Prevent with a global lock order, short critical sections, `tryLock` timeouts; detect with a thread dump (`jstack`, `jcmd <pid> Thread.print`), which reports "Found one Java-level deadlock". Code in Q252. → *More:* §15.9 Liveness · §19.29 Q33

Q159. Livelock vs starvation? Livelock: threads keep reacting to each other and changing state but make no progress (two people stepping aside in a corridor forever). **Starvation**: a thread never gets the CPU or lock because others always win; fair locks or bounded queues help. → *More:* §15.9 Liveness

Q160. ⋆ Why use `ExecutorService` instead of `new Thread()`? What types are there? Reuse, bounded threads, task queue, futures, one place to shut down. Factories: `newFixedThreadPool(n)`, `newCachedThreadPool()`, `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, `newWorkStealingPool()`, `newVirtualThreadPerTaskExecutor()` (21). → *More:* §14.11 Executors · §19.29 Q36

Q161. ⋆ Explain `ThreadPoolExecutor`'s parameters and what happens when a task is submitted. `corePoolSize`, `maximumPoolSize`, `keepAliveTime`, `workQueue`, `threadFactory`, `RejectedExecutionHandler`. On submit: start a new thread if fewer than *core* are running; else queue the task; if the queue is full, start threads up to *max*; if that's full too, **reject** (default `AbortPolicy` throws; alternatives `CallerRunsPolicy`, `DiscardPolicy`, `DiscardOldestPolicy`). Note `newFixedThreadPool` uses an **unbounded** queue — tasks can pile up until out of memory; in production build a `ThreadPoolExecutor` with a bounded queue. → *More:* §14.11 Executors

Q162. How big should a thread pool be? CPU-bound: about the number of cores (`Runtime.getRuntime().availableProcessors()`). I/O-bound: cores × (1 + wait time / compute time), or simply virtual threads with a `Semaphore` around the scarce resource. Measure, don't guess. → *More:* §14.11 Executors

Q163. `shutdown()` vs `shutdownNow()`? `shutdown()` stops accepting new tasks and lets queued ones finish; `shutdownNow()` also interrupts running tasks and returns the queued ones. Follow with `awaitTermination(timeout)`. Since Java 19, `ExecutorService` is `AutoCloseable`, so try-with-resources waits for all tasks. → *More:* §14.11 Executors

Q164. `Runnable` vs `Callable`; `Future` vs `CompletableFuture`? `Callable` returns a value and may throw checked exceptions. `Future.get()` blocks; `CompletableFuture` composes asynchronously. → *More:* §15.2 CompletableFuture · §19.29 Q35

Q165. ⋆ `thenApply` vs `thenCompose` vs `thenCombine`; how are errors handled in `CompletableFuture`? `thenApply` maps the result (like `map`); `thenCompose` chains another async call that returns a `CompletableFuture` (like `flatMap`); `thenCombine` joins two independent futures; `allOf` / `anyOf` wait for many. Errors: `exceptionally` (recover), `handle` (result or error), `whenComplete` (side effect). Without an executor argument, `*Async` methods run on the common `ForkJoinPool` — pass your own executor for blocking I/O. `orTimeout` / `completeOnTimeout` (Java 9) add timeouts. Code in Q255. → *More:* §15.2 CompletableFuture

Q166. What is the Fork/Join framework? A pool for divide-and-conquer tasks (`RecursiveTask` / `RecursiveAction`): split a task, `fork()` one half, compute the other, `join()`. Idle workers **steal** tasks from busy workers' queues. Parallel streams and default `CompletableFuture` async methods use the common pool. → *More:* §15.3 Fork/Join

Q167. ⋆ `CountDownLatch` vs `CyclicBarrier` vs `Semaphore`? `CountDownLatch`: threads wait until a count reaches zero; one-shot (e.g. wait for 3 services to start). `CyclicBarrier`: N threads wait for each other at a point, then all continue; reusable. `Semaphore`: N permits limit how many threads use a resource at once (e.g. at most 10 DB calls). → *More:* §14.13 Synchronizers

Q168. What is a `BlockingQueue`? Which implementations exist? A thread-safe queue where `put` blocks when full and `take` blocks when empty — the standard producer–consumer tool (Q253). `ArrayBlockingQueue` (bounded array), `LinkedBlockingQueue` (optionally bounded), `PriorityBlockingQueue`, `DelayQueue`, `SynchronousQueue` (no capacity: each put waits for a take). → *More:* §14.12 Concurrent collections

Q169. ⋆ What is `ThreadLocal`, and what is its danger? A variable with a separate value per thread (e.g. a `SimpleDateFormat` or the current request's user). In thread pools threads live forever, so values leak or bleed into the next task unless you `remove()` in `finally`. With virtual threads prefer scoped values (Q137). → *More:* §15.6

Q170. What is a daemon thread? A background thread that doesn't keep the JVM alive: when only daemon threads remain, the JVM exits without running their `finally` blocks. Set with `setDaemon(true)` before `start()`. Virtual threads are always daemon threads. → *More:* §14.2 Creating threads

Q171. How do you design a thread-safe class? Prefer immutability; otherwise confine state (one thread owns it), guard every access to shared mutable state with the same lock, use thread-safe building blocks (`ConcurrentHashMap`, atomics), don't let `this` escape the constructor, and document the policy (`@GuardedBy`). Compound actions (check-then-act) need one atomic step. → *More:* §14.14 Designing thread-safe classes

12. JVM, memory and garbage collection

Q172. ⋆ What are the JVM memory areas? Heap vs stack? Heap (objects, shared, GC'd), one stack per thread (frames, locals), metaspace (class metadata, native memory), code cache, direct/native memory. The stack is per thread, LIFO, freed automatically on return, and holds primitives and references; objects themselves live on the heap (unless escape analysis removes the allocation). → *More:* §16.3 · §19.29 Q37

Q173. ⋆ What replaced PermGen, and why? Java 8 replaced PermGen (a fixed-size heap region for class metadata, prone to `OutOfMemoryError: PermGen space`) with **Metaspace** in native memory, which grows as needed (cap it with `-XX:MaxMetaspaceSize`). Interned strings and static fields had already moved to the heap in Java 7. → *More:* §16.3 Memory areas

Q174. ⋆ How does garbage collection work? Which collector is the default? Objects not reachable from GC roots are reclaimed; generational collectors collect the young generation often and the old one rarely. G1 is the default on typical servers since Java 9; on a machine or container with one CPU or under about 1.8 GB of memory the JVM picks Serial instead (still true in 25; JEP 523 makes G1 the default everywhere in a later release). Parallel suits throughput; ZGC (generational only since Java 24) and Shenandoah target sub-millisecond pauses. → *More:* §16.5 GC algorithms · §19.29 Q38

Q175. What are GC roots, and how does an object become eligible for GC? Roots are references the JVM always considers live: local variables on thread stacks, static fields, active threads, JNI references. An object is eligible once no chain of references from a root reaches it — even if objects reference each other in a cycle. → *More:* §16.5 GC algorithms

Q176. Explain young generation, old generation, minor and major GC. New objects go to **Eden**; a **minor GC** copies survivors to a **survivor** space, and objects that survive several collections are **promoted** to the **old generation**. A **major/full GC** collects the old generation (and usually everything), typically with longer pauses. The design relies on the observation that most objects die young. → *More:* §16.5 GC algorithms

Q177. Can you force garbage collection? No. `System.gc()` is only a request (and can be disabled with `-XX:+DisableExplicitGC`). Calling it in application code is almost always wrong. → *More:* §16.5 GC algorithms

Q178. Strong, soft, weak and phantom references? **Strong**: normal; never collected while reachable. **Soft**: cleared only when memory is short (memory-sensitive caches). **Weak**: cleared at the next GC once only weakly reachable (`WeakHashMap`). **Phantom**: only tells you (through a `ReferenceQueue`) that the object was collected; used for clean-up, as `Cleaner` does.

Q179. Name the types of `OutOfMemoryError` and `StackOverflowError` causes. `Java heap space` (too many live objects or a leak), `Metaspace` (too many classes, class-loader leak), `GC overhead limit exceeded` (GC running nearly all the time), `unable to create native thread`, `Direct buffer memory`. `StackOverflowError` comes from too-deep recursion (usually infinite recursion); `-Xss` sets the stack size. → *More:* §16.3 Memory areas

Q180. ⋆ Can Java have memory leaks? How do you find one? Yes — anything still referenced: unbounded caches, static collections, listeners, `ThreadLocal`s, class-loader leaks. Watch heap-after-GC grow, then

use `jcmd GC.class_histogram`, JFR, or a heap dump in Eclipse MAT. → *More:* §16.11 Memory leaks · §19.29 Q39

Q181. ⋆ What is a class loader? Explain parent delegation. A class loader turns `.class` bytes into `Class` objects. Built in: **bootstrap** (core `java.*`), **platform** (other JDK modules; replaced the "extension" loader in Java 9), **application/system** (your class path). A loader first asks its parent; only if the parent can't find the class does it load it itself — so nobody can replace `java.lang.String`. A class is identified by its name **and** its loader. → *More:* §16.2 Class loading · §19.29 Q40

Q182. What are the phases of class loading? Loading (find bytes, create `Class`), **linking** — *verification* (bytecode is safe), *preparation* (static fields get default values), *resolution* (symbolic references → real ones) — and **initialisation** (static initialisers run), which happens lazily on first active use. → *More:* §16.2 Class loading

Q183. What is the JIT compiler? The just-in-time compiler turns frequently executed ("hot") bytecode into native machine code at run time. HotSpot uses tiered compilation: C1 compiles quickly with light optimisation, C2 later with heavy optimisation (inlining, escape analysis, loop unrolling), using profiling data gathered while interpreting. → *More:* §16.8 JIT compilation

Q184. Which JVM flags do you know? `-Xms` / `-Xmx` (initial/max heap), `-Xss` (thread stack), `-XX:+UseG1GC` / `UseZGC` / `UseParallelGC`, `-XX:MaxGCPauseMillis`, `-XX:+HeapDumpOnOutOfMemoryError`, `-Xlog:gc*` (GC logging), `-XX:MaxRAMPercentage` (heap as % of container memory), `-XX:+UseCompactObjectHeaders` (Java 25). → *More:* §16.7 GC tuning basics

Q185. How do you choose a garbage collector? G1 (default) balances throughput and pauses for most services. Parallel GC for batch jobs where total throughput matters and pauses don't. ZGC (generational) or Shenandoah for large heaps and latency-sensitive services needing pauses around a millisecond. Serial for tiny heaps or single-CPU containers. → *More:* §16.6 Collectors compared

Q186. What tools do you use to diagnose a running JVM? `jcmd` (thread dumps, heap dumps, GC info, JFR control), `jstack` (thread dump), `jmap` / `jcmd GC.heap_dump`, `jstat` (GC statistics), JDK Flight Recorder + JDK Mission Control (low-overhead profiling), VisualVM, async-profiler, and Eclipse MAT for heap dumps. → *More:* §16.10 Profiling and diagnostic tools

Q187. What is escape analysis? A JIT analysis that proves an object never leaves a method or thread; the JIT can then skip the heap allocation (scalar replacement — keep the fields in registers) and remove locks on it. → *More:* §16.8 JIT and escape analysis

13. I/O, serialization, reflection and annotations

Q188. Byte streams vs character streams? Why buffer? `InputStream` / `OutputStream` move raw bytes (images, binary); `Reader` / `Writer` move characters and need a **charset** to convert (always specify UTF-8). `BufferedReader` / `BufferedOutputStream` batch many small reads/writes into few system calls — often 10–100× faster. → *More:* §13.5 Byte and character streams · §13.6 Buffering

Q189. `java.io` vs NIO / NIO.2? `java.io` is stream-based and blocking (`File`, `FileInputStream`). NIO (Java 4) added buffers, channels and selectors for non-blocking I/O. NIO.2 (Java 7) added `Path` and `Files` — the modern API: `Files.readString`, `Files.lines`, `Files.walk`, better errors, symbolic links, file attributes. → *More:* §13.1 java.io vs NIO.2

Q190. How do you read a large file line by line efficiently? `try (Stream<String> lines = Files.lines(path, UTF_8)) { ... }` or `BufferedReader.readLine()` in a loop — both stream the file rather than loading it all. Use `Files.readString` / `readAllLines` only for small files. → *More:* §13.4 Reading and writing whole files

Q191. ⋆ What is serialization? What is `serialVersionUID`? Converting an object graph to bytes (`ObjectOutputStream`) and back (`ObjectInputStream`); the class must implement `Serializable`. `serialVersionUID` is the class's version number: if the stream's value differs from the class's, deserialization fails with `InvalidClassException`. Declare it explicitly, otherwise the compiler-computed default changes with almost any class edit. → *More:* §13.10 Serialization

Q192. What does `transient` do? Are static fields serialized? A `transient` field is skipped and comes back as its default value (`null`, 0). Static fields belong to the class, not the object, so they're never serialized. → *More:* §13.10 Serialization

Q193. `Serializable` vs `Externalizable`? What if a parent isn't serializable? `Serializable` is automatic (reflection-based); `Externalizable` makes you write `writeExternal`/`readExternal` yourself and requires a public no-arg constructor. If a superclass isn't serializable, its fields aren't saved and its no-arg constructor runs during deserialization (it must exist).

Q194. Why is Java serialization considered dangerous? How do singletons survive it? Deserializing untrusted bytes can execute "gadget chains" of existing classes — a classic remote-code-execution hole; it also bypasses constructors and ties you to class internals. Prefer JSON/Protobuf, and if you must, use deserialization filters (`ObjectInputFilter`). A singleton needs `readResolve()` returning the instance, or simply use an enum. → *More:* §13.10 Serialization

Q195. What is reflection? Pros and cons? The ability to inspect and use classes at run time: `Class.forName`, `getDeclaredMethods`, `Field.setAccessible(true)`, `Method.invoke`. Frameworks (Spring, Hibernate, JUnit, Jackson) depend on it. Cons: slower, no compile-time checks, breaks encapsulation, and module boundaries restrict it.

Q196. How do annotations work? What is retention? Annotations are metadata on code elements. `@Retention` says how long they live: `SOURCE` (dropped by compiler, e.g. `@Override`), `CLASS` (in the class file, not at run time — the default), `RUNTIME` (readable by reflection, e.g. `@Test`, Spring's annotations). `@Target` limits where they may appear. A custom one: `@Retention(RUNTIME) @Target(METHOD) @interface Timed {}`.

Q197. How many ways can you create an object? `new`, reflection (`clazz.getDeclaredConstructor().newInstance()`), `clone()`, deserialization, and factory methods / builders (which use `new` underneath). Records and enums add their own restrictions (enums can't be instantiated reflectively).

14. Testing and tooling

Q198. ⋆ How do you write a unit test in JUnit (5/6)? Annotate a method with `@Test`, arrange inputs, act, assert with `assertEquals`/`assertTrue` or AssertJ's `assertThat(x).isEqualTo(y)`. Lifecycle: `@BeforeEach`/`@AfterEach` around each test, `@BeforeAll`/`@AfterAll` (static) once per class; JUnit 4 called these `@Before`/`@BeforeClass`. The book uses JUnit 6, which keeps the JUnit 5 ("Jupiter") API. → *More:* §18.4 JUnit

Q199. How do you test that a method throws an exception? `var ex = assertThrows(IllegalArgumentException.class, () -> account.withdraw(-1)); assertEquals("negative amount", ex.getMessage());` or AssertJ `assertThatThrownBy(...).isInstanceOf(...).hasMessage(...)`. → *More:* §18.4 JUnit

Q200. ⋆ What is mocking? Mock vs stub vs spy? A **test double** replaces a real dependency. A **stub** returns canned answers; a **mock** also lets you verify interactions (`verify(repo).save(order)`); a **spy** wraps a real object and calls real methods unless stubbed. With Mockito: `@Mock`, `@InjectMocks`, `when(x.f()).thenReturn(v)`, `ArgumentCaptor`. Static methods can be mocked with `mockStatic` (Mockito 5's default inline mock maker), though needing it is often a design smell. → *More:* §18.6 Test doubles and Mockito

Q201. What are parameterized tests? One test method run with many inputs: `@ParameterizedTest` with `@ValueSource`, `@CsvSource`, `@MethodSource` or `@EnumSource`. → *More:* §18.5 Parameterized tests

Q202. Unit vs integration vs end-to-end tests; what is TDD? Unit: one class in isolation, fast. Integration: several real components together (database via Testcontainers, HTTP layer). End-to-end: the whole system as a user sees it. Keep many unit tests, fewer integration, few E2E (the test pyramid). **TDD** (test-driven development): write a failing test, make it pass, refactor. → *More:* §18.9 Test design

Q203. Maven vs Gradle? What are the Maven lifecycle phases? Both resolve dependencies and build. Maven: XML `pom.xml`, fixed conventions, lifecycle `validate → compile → test → package → verify → install → deploy`. Gradle: Kotlin/Groovy build scripts, incremental builds and build cache, more flexible and usually faster. Dependency scopes like `test` or `provided` keep libraries out of the runtime. → *More:* §18.2 Maven · §18.3 Gradle

Q204. How should you log in Java? Through the SLF4J API with Logback or Log4j 2 behind it; use parameterised messages (`log.info("user {} logged in", id)`) so strings are built only when the level is enabled; never log secrets; log an exception once, with its stack trace. → *More:* §18.10 Logging

15. Scenario and senior-level questions

Q205. ⋆ Your service's CPU is at 100%. How do you investigate? Find the hot threads (`top -H -p <pid>`, convert the thread ID to hex) and match them in a thread dump (`jcmd <pid> Thread.print`), taking several dumps a few seconds apart. Check GC logs — constant GC from a nearly full heap looks like high CPU. Profile with JFR or async-profiler for a flame graph. Typical culprits: infinite loops, regex backtracking, busy-wait spin loops, and GC thrashing. → *More:* §16.10 Diagnostic tools

Q206. ⋆ The application throws `OutOfMemoryError` in production. What do you do? Read the message (heap, Metaspace, native thread…). Make sure `-XX:+HeapDumpOnOutOfMemoryError` is on, open the dump in Eclipse MAT and look at the dominator tree / biggest retained sizes. Check whether heap-after-GC grows steadily (leak) or the heap is simply too small for the load. Fix the leak or bound the cache before raising `-Xmx`. → *More:* §16.11 Memory leaks

Q207. Requests hang and threads pile up. What would you look at? Thread dumps: are many threads `BLOCKED` on one lock, `WAITING` on a connection pool, or stuck in socket reads with no timeout? Typical fixes: set timeouts on every remote call, size or bound pools, remove the lock contention, and look for a deadlock reported in the dump. → *More:* §16.10 Diagnostic tools

Q208. How would you make a `HashMap`-based cache thread-safe and bounded? Use `ConcurrentHashMap` with `computeIfAbsent` for atomic load-on-miss, and add eviction (size or time based) — a hand-rolled version gets hard quickly, so in production use Caffeine. For an LRU cache with a lock, wrap a `LinkedHashMap` (Q249) with `synchronized` methods.

Q209. Why can a `parallelStream()` make a web service slower? All parallel streams share the JVM-wide common `ForkJoinPool` (sized cores − 1). Under many concurrent requests they compete for the same few threads, and blocking calls inside them stall the pool for everyone. In servers, prefer sequential streams, or an explicit executor for parallel work. → *More:* §12.8 Parallel streams

16. "What does this print?" output puzzles

All outputs below were produced by running the code.

Q210. ⋆ String pool and `==`.

```
String s1 = "Java", s2 = "Ja" + "va", s3 = new String("Java");
String part = "Ja", s4 = part + "va";
final String fp = "Ja";
System.out.println((s1 == s2) + " " + (s1 == s3) + " " + (s1 == s4) + " "
       + (s1 == s4.intern()) + " " + (s1 == fp + "va"));
```

Prints `true false false true true`. Literal-only concatenation (and a `final` constant) is folded at compile time into the pooled `"Java"`; `new` and run-time concatenation create new objects; `intern()` returns the pooled one. → *More:* §7.1 Strings and the pool

Q211. ⋆ The `Integer` cache.

```
Integer a = 127, b = 127, c = 128, d = 128;
System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
```

Prints `true false true`. Autoboxing calls `Integer.valueOf`, which caches −128…127; 128 gives two different objects. Compare wrappers with `equals`. → *More:* §7.6 Wrappers and autoboxing

Q212. ⋆ `return` in `try` and `finally`.

```
static int f1() { try { return 1; } finally { return 2; } }
static int f2() { int x = 1; try { return x; } finally { x = 2; } }
```

`f1()` returns `2` — the `finally` return replaces the first one. `f2()` returns `1` — the return value was already evaluated before `finally` changed `x`. → *More:* §8.4 try/catch/finally

Q213. String concatenation order. `System.out.println(1 + 2 + "3" + 4 + 5);` prints `3345`. `+` is left-associative: `1 + 2` is int addition (3), then everything after the first string is concatenation. → *More:* §7.1 Strings and the pool

Q214. `char` **arithmetic.** `System.out.println('a' + 'b');` prints `195` (both promoted to `int`: 97 + 98), but `"" + 'a' + 'b'` prints `ab`. And `char ch = 'a'; ch += 1;` gives `b` (compound assignment casts back), while `System.out.println(ch + 1)` with the original `ch = 'a'` prints `98` (an `int`).

Q215. Floating point and overflow. `0.1 + 0.2 == 0.3` is `false` (sum is `0.30000000000000004`). `Math.abs(Integer.MIN_VALUE)` is `-2147483648` (its positive value doesn't fit in an `int`), and `Integer.MAX_VALUE + 1` is `-2147483648`. → *More:* §2.11 Floating point · §2.10 Integer overflow

Q216. ⋆ Overloading with `null`.

```
static void m(Object o) { System.out.println("Object"); }
static void m(String s) { System.out.println("String"); }
m(null);
```

Prints `String`: the compiler picks the **most specific** applicable overload. Add `m(Integer)` too and the call becomes ambiguous — a compile error. → *More:* §4.3 Overloading

Q217. ⋆ Overriding vs hiding (fields and static methods).

```
class P { String name = "P"; static String s() { return "P.s"; } String who() { return "P.who"; } }
class C extends P { String name = "C"; static String s() { return "C.s"; } String who() { return "C.who"; } }
P p = new C();
System.out.println(p.name + " " + p.s() + " " + p.who());
```

Prints `P P.s C.who`. Fields and static methods are resolved from the **declared** type (`P`); only instance methods are dispatched on the run-time object. → *More:* §6.2 Hiding

Q218. ⋆ Calling an overridable method from a constructor.

```
class P { P() { init(); } void init() {} }
class C extends P { String v = "set"; void init() { System.out.println("init sees v=" + v); } }
new C();
```

Prints `init sees v=null`. `P`'s constructor runs before `C`'s field initialisers, but `init()` dispatches to `C`. Never call overridable methods from constructors. → *More:* §5.10 Initialisation order

Q219. `i = i++`. `int i = 0; i = i++;` leaves `i` at `0`: `i++` yields the old value (0), increments `i` to 1, then the assignment writes the old value 0 back. Similarly `int x = 5; x += x++ * 2;` gives `15` (5 + 5·2, with the left `x` read before the increment).

Q220. `List.remove(int)` **vs** `remove(Object)`.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
list.remove(1);                    // index 1 → [1, 3]
list.remove(Integer.valueOf(1));   // the value 1 → [3]
```

An `int` argument picks `remove(int index)` because exact match beats boxing.

Q221. `Arrays.asList` **traps.** `Arrays.asList(new int[]{1, 2, 3}).size()` is `1` — a primitive array is one object, giving a `List<int[]>`; `Arrays.asList(1, 2, 3).size()` is `3`. And `Arrays.asList(3, 1, 2).add(4)` throws `UnsupportedOperationException` (fixed-size view).

Q222. ⋆ Removing inside a for-each loop.

```
List<String> l = new ArrayList<>(List.of("a", "b", "c"));
for (String x : l) if (x.equals("b")) l.remove(x);
System.out.println(l);
```

Prints `[a, c]` — **no** exception, because removing the second-to-last element shrinks the list so `hasNext()` returns false before the check runs. Removing `"a"` instead throws `ConcurrentModificationException`. Either way, use `l.removeIf(...)`. → *More:* §10.7 Iterators

Q223. ⋆ Stream laziness and short-circuiting.

```
Stream.of(1, 2, 3).peek(x -> System.out.println("never")).map(x -> x * 2);
String r = Stream.of("a", "bb", "ccc")
        .filter(x -> { System.out.print(x + " "); return x.length() > 1; })
        .findFirst().get();
```

The first line prints nothing (no terminal operation). The second prints `a bb` and returns `bb`: `findFirst` stops the pipeline, so `"ccc"` is never examined. → *More:* §12.2 Laziness

Q224. ★ `equals` without `hashCode`.

```
class Key { int k; Key(int k) { this.k = k; }
    public boolean equals(Object o) { return o instanceof Key x && x.k == k; } }
Set<Key> set = new HashSet<>(); set.add(new Key(1)); set.add(new Key(1));
System.out.println(set.size());
```

Prints `2` even though the keys are equal: each has a different identity hash code, so they land in different buckets. → *More:* §6.13 equals and hashCode

Q225. Ternary unboxing NPE.

```
Integer n = null;
int y = flag ? n : 0;   // NullPointerException when flag is true
```

With one `Integer` and one `int` operand, the result type is `int`, so `n` is unboxed.

Q226. Narrowing casts. `(byte) 200` is `-56` (keeps the low 8 bits), `(int) 3.99` is `3` (truncates toward zero), `(char) 65` is `A`.

Q227. Strings are immutable. `String s = "hello"; s.toUpperCase(); System.out.println(s);` prints `hello`. The result was discarded — write `s = s.toUpperCase();`.

Q228. try-with-resources close order.

```
try (R r1 = new R("first"); R r2 = new R("second")) { System.out.println("body"); }
```

Prints `body`, `close second`, `close first` — resources close in reverse order of declaration. → *More:* §8.6 try-with-resources

Q229. Division edge cases. `10 / 0` throws `ArithmeticException: / by zero`; `10.0 / 0` is `Infinity`; `0.0 / 0` is `NaN`; `Double.NaN == Double.NaN` is `false` (use `Double.isNaN`). `Math.round(-2.5)` is `-2` and `Math.round(2.5)` is `3` (it rounds half up: `floor(x + 0.5)`). → *More:* §2.11 Floating point

Q230. Overload resolution order: widening, boxing, varargs. With `m2(long)` and `m2(Integer)`, `m2(5)` prints `long`. With `m3(long)` and `m3(int...)`, `m3(5)` prints `long3`. The compiler tries, in phases: exact/widening first, then boxing, then varargs. → *More:* §4.3 Overloading

Q231. Immutable factory traps. `List.of("a", null)` throws `NullPointerException`; `Map.of("k", 1, "k", 2)` throws `IllegalArgumentException` (duplicate key).

Q232. `run()` instead of `start()`. `new Thread(() -> System.out.println(Thread.currentThread().getName())).run();` prints `main` — no new thread was started.

Q233. `HashMap` and the null key. `map.put(null, 1); map.put(null, 2);` leaves `{null=2}`. `put` returns the previous value, so `map.put("z", 3)` on an absent key returns `null`.

Q234. `StringBuilder` equality. `new StringBuilder("x").equals(new StringBuilder("x"))` is `false` — `StringBuilder` doesn't override `equals`. Compare `sb1.toString().equals(sb2.toString())` or use `sb1.compareTo(sb2) == 0` (Java 11).

Q235. `BigDecimal` in `HashSet` vs `TreeSet`. Adding `new BigDecimal("1.0")` and `new BigDecimal("1.00")`: a `HashSet` has size `2` (`equals` compares scale too), a `TreeSet` has size `1` (`compareTo` says they're equal). Also `new BigDecimal(0.1)` prints `0.1000000000000000055511151231257827021181583404541015625`, while `BigDecimal.valueOf(0.1)` prints `0.1`.

Q236. `switch` on a null string. `String s = null; switch (s) { case "a" -> ...; default -> ...; }` throws `NullPointerException` — `default` doesn't match `null`. Since Java 21 you can add `case null ->`.

Q237. Overflow before widening. `long big = Integer.MAX_VALUE * 2;` is `-2`: the multiplication happens in `int` and overflows *before* the assignment widens it. `Integer.MAX_VALUE * 2L` gives `4294967294`. → *More:*

§2.10 Integer overflow

Q238. Integer literal prefixes. `System.out.println(010 + 0x10 + 0b10);` prints `26`: octal 8 + hex 16 + binary 2.

Q239. `split` drops trailing empty strings. `"a,b,,".split(",").length` is `2`. Use `split(",", -1)` to keep them (length 4).

17. Coding tasks asked in Java rounds

These are the "write it in Java" tasks that keep appearing in Java-developer interviews (especially Java 8 stream tasks). Each solution was run; the result is shown in the comment.

Q240. ⋆ Reverse a string.

```
String s = "programming";
String r1 = new StringBuilder(s).reverse().toString();   // gnimmargorp
String r2 = "";
for (int i = s.length() - 1; i >= 0; i--) r2 += s.charAt(i);        // O(n²): mention it
```

Interviewers often forbid `reverse()`: swap `char`s in a `char[]` with two pointers instead (O(n)). → *More:* §19.2

Q241. Check whether a string is a palindrome.

```
boolean pal = IntStream.range(0, w.length() / 2)
        .allMatch(i -> w.charAt(i) == w.charAt(w.length() - 1 - i));  // "racecar" → true
```

For "ignore case and punctuation" variants, skip non-letters with two pointers and compare `Character.toLowerCase`.

Q242. Check whether two strings are anagrams.

```
char[] a = "listen".toCharArray(), b = "silent".toCharArray();
Arrays.sort(a); Arrays.sort(b);
boolean anagram = Arrays.equals(a, b);   // true
```

O(n log n). O(n): count letters in an `int[26]`, increment for one string, decrement for the other, check all zeros.

Q243. ⋆ Count character frequency and find the first non-repeated character (streams).

```
Map<Character, Long> freq = "programming".chars().mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
char first = freq.entrySet().stream().filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey).findFirst().orElseThrow();   // p
```

`LinkedHashMap::new` keeps first-appearance order — that's what makes "first" correct. → *More:* §19.3 HashMap idioms

Q244. Find duplicate elements.

```
Set<Character> seen = new HashSet<>();
Set<Character> dups = "programming".chars().mapToObj(c -> (char) c)
        .filter(c -> !seen.add(c))
        .collect(Collectors.toCollection(LinkedHashSet::new));   // [r, m, g]
```

`Set.add` returns `false` for an element already present. Alternative: `groupingBy` + `counting()`, keep counts > 1. (The stateful `filter` is fine sequentially, not in a parallel stream.) → *More:* §19.3 HashMap idioms

Q245. ⋆ Find the second-highest number.

```
int second = List.of(5, 9, 2, 9, 7, 2, 4).stream().distinct()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst().orElseThrow();   // 7
```

`distinct()` matters: without it the answer would be the duplicate 9. In a loop, track `max` and `second` in one O(n) pass.

Q246. ⋆ Employee stream tasks: count per department, average salary, top earner per department.

```
record Employee(String name, String dept, double salary, int age) {}
var emps = List.of(new Employee("Ana", "Eng", 120, 28), new Employee("Bo", "Eng", 100, 35),
        new Employee("Cy", "Ops", 90, 40), new Employee("Di", "Ops", 95, 25), new Employee("Ed", "HR", 70, 45));
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new, counting()));
// {Eng=2, HR=1, Ops=2}
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new, averagingDouble(Employee::salary)));
// {Eng=110.0, HR=70.0, Ops=92.5}
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        collectingAndThen(maxBy(comparingDouble(Employee::salary)), o -> o.orElseThrow().name())));
// {Eng=Ana, HR=Ed, Ops=Di}
emps.stream().filter(e -> e.age() > 30).sorted(comparingDouble(Employee::salary).reversed())
        .map(Employee::name).collect(joining(", "));   // Bo, Cy, Ed
```

(Static imports from `Collectors` and `Comparator`.) Second-highest salary: `map(Employee::salary).distinct().sorted(reverseOrder()).skip(1).findFirst()`. → *More:* §12.5 Collectors

Q247. ⋆ Sort a map by its values.

```
Map<String, Integer> scores = Map.of("a", 3, "b", 1, "c", 2);
Map<String, Integer> sorted = scores.entrySet().stream()
        .sorted(Map.Entry.comparingByValue())
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));   // {b=1, c=2, a=3}
```

The `LinkedHashMap` supplier is essential — a plain `toMap` returns a `HashMap` and loses the order.

Q248. ⋆ Write a thread-safe singleton.

```
final class Config {                          // holder idiom: lazy, thread-safe, no locking
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }
    static Config get() { return Holder.INSTANCE; }
}
final class Dcl {                             // double-checked locking
    private static volatile Dcl instance;     // volatile is mandatory
    static Dcl get() {
        Dcl r = instance;
        if (r == null) synchronized (Dcl.class) { r = instance; if (r == null) instance = r = new Dcl(); }
        return r;
    }
}
```

Without `volatile`, another thread could see a non-null reference to a not-yet-fully-constructed object. Simplest of all: `enum Config { INSTANCE; }`.

Q249. ⋆ Implement an LRU cache.

```
class LRU<K, V> extends LinkedHashMap<K, V> {
    private final int cap;
    LRU(int cap) { super(16, 0.75f, true); this.cap = cap; }   // true = access order
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > cap; }
}
// cap 2: put 1, put 2, get 1, put 3 → keys [1, 3]
```

If asked to do it "from scratch", describe `HashMap<K, Node>` + doubly linked list (Q102). → *More:* §10.4 Maps

Q250. ⋆ Write an immutable class.

```java
final class Money {
    private final String currency;
    private final List<String> tags;
    Money(String currency, List<String> tags) {
        this.currency = Objects.requireNonNull(currency);
        this.tags = List.copyOf(tags);          // defensive, unmodifiable copy
    }
    String currency() { return currency; }
    List<String> tags() { return tags; }       // safe to return: can't be modified
}
```

Then point out a `record Money(String currency, List<String> tags)` with `tags = List.copyOf(tags)` in a compact constructor does the same. → *More:* §5.9 Immutability

Q251. Implement a simple `HashMap`.

```java
class SimpleHashMap<K, V> {
    private static class Node<K, V> { final K key; V value; Node<K, V> next;
        Node(K k, V v, Node<K, V> n) { key = k; value = v; next = n; } }
    private Node<K, V>[] table = newTable(16);
    private int size;
    private static int index(Object key, int n) { int h = Objects.hashCode(key); return (h ^ (h >>> 16)) & (n -
1); }
    public V get(K key) {
        for (Node<K, V> e = table[index(key, table.length)]; e != null; e = e.next)
            if (Objects.equals(e.key, key)) return e.value;
        return null;
    }
    public void put(K key, V value) {
        int i = index(key, table.length);
        for (Node<K, V> e = table[i]; e != null; e = e.next)
            if (Objects.equals(e.key, key)) { e.value = value; return; }
        table[i] = new Node<>(key, value, table[i]);
        if (++size > table.length * 3 / 4) resize();   // rehash every node into a 2× table
    }
    // newTable: @SuppressWarnings("unchecked") cast of new Node[n]; resize: loop over old buckets
}
```

Talk through: power-of-two capacity, hash spreading, chaining, load factor 0.75, `equals` for key match, null key via `Objects.hashCode`. (The full version with `resize` was run: 100 keys in, updates and lookups correct.) → *More:* §10.5 HashMap internals

Q252. ⋆ Write a program that deadlocks — then fix it.

```java
Object a = new Object(), b = new Object();
new Thread(() -> { synchronized (a) { sleep(100); synchronized (b) { } } }).start();
new Thread(() -> { synchronized (b) { sleep(100); synchronized (a) { } } }).start();
// both threads end up BLOCKED forever
```

Fix: make both threads take `a` then `b` (one global lock order), or use `ReentrantLock.tryLock(timeout)` and back off. → *More:* §15.9 Liveness

Q253. ⋆ Producer–consumer with a `BlockingQueue`.

```java
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(2);
Thread producer = Thread.ofPlatform().start(() -> {
    try { for (int v = 1; v <= 5; v++) q.put(v); q.put(-1); }        // -1 = "poison pill" end marker
    catch (InterruptedException e) { Thread.currentThread().interrupt(); } });
Thread consumer = Thread.ofPlatform().start(() -> {
    try { for (int v; (v = q.take()) != -1; ) got.add(v); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); } });
// got = [1, 2, 3, 4, 5]
```

`put` blocks when full, `take` when empty, so no explicit locking is needed. → *More:* §14.12 Concurrent collections

Q254. Producer–consumer with `wait` / `notify` (a bounded buffer).

```
final class BoundedQueue<T> {
    private final Deque<T> items = new ArrayDeque<>();
    private final int cap;
    BoundedQueue(int cap) { this.cap = cap; }
    synchronized void put(T t) throws InterruptedException {
        while (items.size() == cap) wait();      // while, not if: re-check after waking
        items.addLast(t); notifyAll();
    }
    synchronized T take() throws InterruptedException {
        while (items.isEmpty()) wait();
        T t = items.removeFirst(); notifyAll(); return t;
    }
}
```

→ *More:* §14.8 wait/notify

Q255. ⋆ Run two async calls and combine them with `CompletableFuture`.

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    var price = CompletableFuture.supplyAsync(() -> 100, ex);
    var rate  = CompletableFuture.supplyAsync(() -> 2, ex);
    int total = price.thenCombine(rate, (p, r) -> p * r)
            .orTimeout(1, TimeUnit.SECONDS)
            .exceptionally(t -> -1)       // fallback on failure or timeout
            .join();                      // 200
}
```

For N calls: `CompletableFuture.allOf(fs).thenApply(v -> fs.stream().map(CompletableFuture::join).toList())`.

Q256. ⋆ Print odd and even numbers alternately with two threads.

```
static void printTurn(Object lock, int[] counter, int parity, int max) {
    synchronized (lock) {
        while (counter[0] <= max) {
            if (counter[0] % 2 == parity) { System.out.print(counter[0]++ + " "); lock.notifyAll(); }
            else try { lock.wait(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); return;
}
        }
        lock.notifyAll();   // release the other thread so it can exit
    }
}
// thread A: printTurn(lock, counter, 1, 10); thread B: printTurn(lock, counter, 0, 10)
// 1 2 3 4 5 6 7 8 9 10
```

Alternatives: two `Semaphore`s handing a permit back and forth, or a `ReentrantLock` with two `Condition`s.
→ *More:* §14.8 wait/notify

Q257. Count words; flatten a list of lists.

```
Map<String, Long> wc = Arrays.stream("to be or not to be".split(" "))
        .collect(Collectors.groupingBy(Function.identity(), TreeMap::new, Collectors.counting()));
// {be=2, not=1, or=1, to=2}
List<Integer> flat = List.of(List.of(1, 2), List.of(3), List.<Integer>of())
        .stream().flatMap(List::stream).toList();   // [1, 2, 3]
```

Loop version of word count: `counts.merge(word, 1, Integer::sum)`. → *More:* §12.5 Collectors

Q258. Fibonacci and primes.

```
Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
        .limit(10).map(f -> f[0]).toList();         // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
IntStream.rangeClosed(2, 30)
        .filter(n -> IntStream.rangeClosed(2, (int) Math.sqrt(n)).noneMatch(d -> n % d == 0))
        .boxed().toList();                          // [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Mention: recursive Fibonacci without memoisation is $O(2^n)$; the iterative one is $O(n)$. For factorials beyond 20!, `long` overflows — use `BigInteger` (25! = 15511210043330985984000000).

Q259. Merge two sorted arrays; find the missing number in 1..n.

```
int i = 0, j = 0, k = 0;
while (i < a.length && j < b.length) m[k++] = a[i] <= b[j] ? a[i++] : b[j++];
while (i < a.length) m[k++] = a[i++];
while (j < b.length) m[k++] = b[j++];      // {1,3,5} + {2,4,6,8} → [1, 2, 3, 4, 5, 6, 8]
int missing = n * (n + 1) / 2 - Arrays.stream(arr).sum();   // {1,2,4,5}, n=5 → 3
```

For large n the sum can overflow — use `long` or XOR all values and indices.

Q260. String compression (`aaabccdddd` → `a3b1c2d4`).

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < s.length(); ) {
    int j = i;
    while (j < s.length() && s.charAt(j) == s.charAt(i)) j++;
    sb.append(s.charAt(i)).append(j - i);
    i = j;
}
return sb.length() < s.length() ? sb.toString() : s;   // a3b1c2d4
```

Q261. Reverse the words of a sentence; count vowels.

```
String.join(" ", Arrays.asList("the quick brown fox".split("\\s+")).reversed());  // fox brown quick the
"Interview".toLowerCase().chars().filter(c -> "aeiou".indexOf(c) >= 0).count();    // 4
```

`List.reversed()` is a Java 21 sequenced-collection view; before 21 use `Collections.reverse`.

Q262. Swap two numbers without a temporary variable. `a = a ^ b; b = a ^ b; a = a ^ b;` (3, 7 → 7, 3). The arithmetic version `a = a + b; b = a - b; a = a - b;` can overflow. In real code just use a temp variable.

Q263. Count events from many threads correctly.

```
var counter = new AtomicInteger();
try (var ex = Executors.newFixedThreadPool(4)) {
    for (int t = 0; t < 1000; t++) ex.submit(counter::incrementAndGet);
}                                       // close() waits for all tasks
System.out.println(counter.get());      // 1000 (a plain int++ would usually lose updates)
```

→ *More:* §14.10 Atomic variables

18. DSA problems in Java rounds

Java rounds usually include one or two classic LeetCode problems. Chapter 19 solves each pattern in idiomatic Java; here is the problem, the one-line approach, and where the full solution lives.

Q264. ⋆ Two Sum. One pass with a `HashMap<value, index>`: for each `x`, check whether `target − x` is already in the map. O(n). → §19.10 Hashing

Q265. ⋆ Longest substring without repeating characters. Sliding window: move `right`, and when a character repeats jump `left` past its previous position (stored in a map or `int[128]`). O(n). → §19.9 Sliding window

Q266. Valid parentheses. Push opening brackets on an `ArrayDeque`; on a closing bracket pop and check it matches; valid if the stack ends empty. → §19.12 Stacks

Q267. Reverse a linked list; detect a cycle. Reverse with three pointers `prev` / `cur` / `next`. Cycle: Floyd's fast/slow pointers meet if there is a cycle. → §19.22 Linked lists

Q268. ⋆ K-th largest element / top-k frequent. Keep a min-heap (`PriorityQueue`) of size k; the root is the answer. O(n log k). Quickselect gives O(n) average. → §19.13 Heaps

Q269. Binary search (and its overflow bug). Compute the middle as `lo + (hi - lo) / 2` (or `(lo + hi) >>> 1`); `(lo + hi) / 2` overflows for large indices. → §19.14 Binary search

Q270. ⋆ Merge overlapping intervals. Sort by start (`Comparator.comparingInt(a -> a[0])`), then extend the last merged interval or start a new one. O(n log n). → §19.15 Intervals

Q271. Group anagrams. Key each word by its sorted characters; `computeIfAbsent(key, k -> new ArrayList<>()).add(word)`. → §19.10 Hashing

Q272. Level-order traversal; number of islands. BFS with an `ArrayDeque` queue (process one level per `size()` snapshot); islands = count BFS/DFS starts over unvisited land cells. → §19.16 BFS and DFS

Q273. Maximum subarray, climbing stairs, coin change. Dynamic programming: Kadane's algorithm keeps the best sum ending here; stairs is Fibonacci; coin change fills `dp[amount]` bottom-up. → §19.19 Dynamic programming

Q274. Which Java-specific pitfalls cost points in coding rounds? `==` on `Integer`/`String`, `int` overflow in sums and midpoints, `Arrays.sort` on `int[]` with a comparator (needs `Integer[]`), comparators written as `a - b`, using `Stack`/`LinkedList` instead of `ArrayDeque`, string `+=` in loops, and forgetting `assert` is disabled by default. → §19.6 Pitfalls

19. Multithreading problems (Chapter 22)

Q275. ⋆ Implement a counting semaphore without `java.util.concurrent`. One `int permits` under the object's monitor: `acquire` = `while (permits == 0) wait(); permits--`, `release` = `permits++; notifyAll()`. An interrupted waiter takes nothing; a timed version loops on a fixed `nanoTime` deadline; FIFO fairness needs a queue where only the head may take a permit. → §22.2

Q276. Build a read-write lock with writer preference. State `readers`, `writing`, `waitingWriters`. Readers wait `while (writing || waitingWriters > 0)`; writers increment `waitingWriters`, wait `while (writing || readers > 0)`, and must decrement it again if interrupted. The last reader and every leaving writer call `notifyAll()`. Writer preference can starve readers; the fair `ReentrantReadWriteLock` bounds both. → §22.3

Q277. Why does a naive barrier break when reused, and how do you fix it? Without a reset, round 2 sees `count >= N` and nobody waits; resetting while waiters loop on `count < N` strands them. Use a **generation** number: waiters loop `while (myGeneration == generation)`, and the last arriver resets the count, bumps the generation and calls `notifyAll()`. → §22.4

Q278. Design a deferred callback executor (run each callback after its delay). A min-heap of `(nanoTime deadline, seq, callback)` under a lock and one worker that `awaitNanos(head − now)`. `schedule()` signals only when the new task becomes the head; the worker re-reads the head after every wake-up, so no signal can be missed. Run callbacks outside the lock and catch their exceptions. → §22.5

Q279. ⋆ Turn a callback-based async API into a blocking call. Create a per-call mailbox, let the callback record the outcome *in state* (first completion wins) and wake the caller, who waits `while (!done)` with a timeout. Simplest: complete a `CompletableFuture` in the callback and call `get(timeout)`. Never block on the library's own callback thread. → §22.6

Q280. Three threads call `first()`, `second()`, `third()` in any order; print them in order. Two one-shot gates that remember being opened: a `CountDownLatch(1)` or `Semaphore(0)` per gate, or a monitor with an `int stage` and `while (stage != mine) wait()` + `notifyAll()`. A bare `notify()` sent before the waiter arrives would be lost. → §22.7

Sources of the question lists

Question wording and frequency were taken from these public lists and reports (answers are this book's own):

- teamlead/java-interview-questions (GitHub) — questions ranked by how often they were asked in real interviews (high/moderate/low occurrence)
- learning-zone/java-basics (GitHub) — "Java Basics" interview question collection
- in28minutes/JavaInterviewQuestionsAndAnswers (GitHub) — 200+ Java interview questions list
- Devinterview-io/java-interview-questions (GitHub) — top Java interview questions
- DopplerHQ/awesome-interview-questions (GitHub) — Java section and the lists it links (Toptal "Essential Java Interview Questions", Java Code Geeks "115 Java Interview Questions")
- sats17/Java-8-Interview-Sample-Coding-Questions (GitHub) and the widely shared "commonly asked Java 8 stream coding questions" articles (Medium)
- LeetCode Discuss interview-experience posts for Java back-end roles (e.g. "Senior Software Engineer | Java tech stack": Java 8 features, `HashMap` and `ConcurrentHashMap` internals)

- Glassdoor interview reports for Java developer roles (`HashMap` vs `ConcurrentHashMap`, stream tasks on employee lists)
- LeetCode "Top Interview 150" problem list (for the DSA problems in group 18)